

Computer Vision

Summary Notes

Classical & Geometric Computer Vision

Contents

1	Human Color Perception	1
2	Sensing Light & Color Filters	2
2.1	3-Chip vs. 1-Chip cameras	2
2.2	The Bayer Filter	2
3	Color Spaces	3
3.1	RGB	3
3.2	HSV — Hue, Saturation, Value	3
3.3	YCbCr	3
3.4	L*a*b*	4
4	Image Filtering	4
4.1	2D Correlation vs. 2D Convolution	4
4.1.1	Properties of convolution	4
4.2	Box Filter	4
4.3	Gaussian Filter	5
4.4	Separability	5
4.4.1	Runtime — $N \times N$ image, $K \times K$ kernel	5
4.5	Sobel Filter	5
5	Non-linear Filters & Morphology	6
5.1	Median Filter	6
5.2	Morphological Operations	6
6	Frequency Domain	6
6.1	Nyquist–Shannon Sampling Theorem	6
6.2	Downsampling — Image Pyramid	6
6.3	Fourier Transform	7
6.4	Deconvolution	7
7	JPEG Compression	8
8	Gamma Correction	9
9	Edge Detection & Canny	9
9.1	Canny Edge Detector — the full pipeline	10

10 Corner Detection (Harris)	10
10.1 What makes a good feature?	10
10.2 Corner detection by auto-correlation	10
10.3 Second-order Taylor approximation	11
10.4 Eigenvalues of M	11
10.5 Cornerness response	11
10.6 Invariance vs. covariance	11
11 Features & Matching — SIFT	12
11.1 Step 1 — Find keypoints	12
11.1.1 Post-processing	12
11.2 Step 2 — Orientation assignment	12
11.3 Step 3 — Descriptor extraction	13
11.4 Feature matching — Nearest Neighbour Distance Ratio (NNDR)	14
12 Cameras & Optics	14
12.1 Pinhole Camera Model	14
12.2 Underlying physics	14
13 Homogeneous Coordinates, Points & Lines	15
13.1 Points & lines (2D)	15
14 The Camera Matrix	16
14.1 Intrinsic matrix K — 5 DoF	16
14.2 Extrinsic matrix $[R \mid t]$ — 3 DoF rotation + 3 DoF translation	16
15 Singular Value Decomposition (SVD)	17
15.1 Solving least-squares systems with SVD	17
15.2 Forcing rank-deficiency	17
16 2D Transformations	17
17 Camera Calibration (Direct Linear Transform)	18
17.1 Two ways to solve	18
17.2 Decomposing M back into K, R, t	18
18 Dense Motion Estimation	19
18.1 Optical flow $\neq$ motion flow	19
18.2 The Aperture Problem	19
18.3 Error metrics	19

19 Lucas–Kanade Optical Flow	20
19.1 Basic problem	20
19.2 Patch-wise SSD	20
19.3 Minimization	21
19.4 Iterative solver	21
19.5 Coarse-to-fine (pyramidal LK)	21
20 Epipolar Geometry	22
20.1 Projective geometry in 3D	22
20.1.1 Hessian normal form of a plane	22
20.1.2 3D lines	23
20.2 The projection matrix P	24
20.3 Two-view geometry	25
21 The Fundamental Matrix	25
21.1 Estimating F — the 8-point algorithm	25
22 Stereo Vision	26
22.1 Triangulation	26
22.2 Impact of the baseline	26
22.3 Rectification	27
22.4 Window-based matching	27
23 Structured Light & Time of Flight	28
23.1 Passive vs. active stereo	28
23.2 Triangulation with a light plane	28
23.3 Binary coded light striping	28
23.4 Time of Flight (ToF)	29
24 Surface Reconstruction	29
24.1 Task	29
24.2 Coarse alignment — Procrustes	29
24.3 Fine alignment — Iterative Closest Point (ICP)	29
24.3.1 Point-to-point vs. point-to-plane	30
24.4 Implicit functions	30
24.5 Marching Squares / Marching Cubes	31
24.6 Hoppe’s method — point cloud to implicit function	32
24.7 Truncated Signed Distance Fields (TSDF)	32

1 Human Color Perception

The retina contains two classes of photoreceptors:

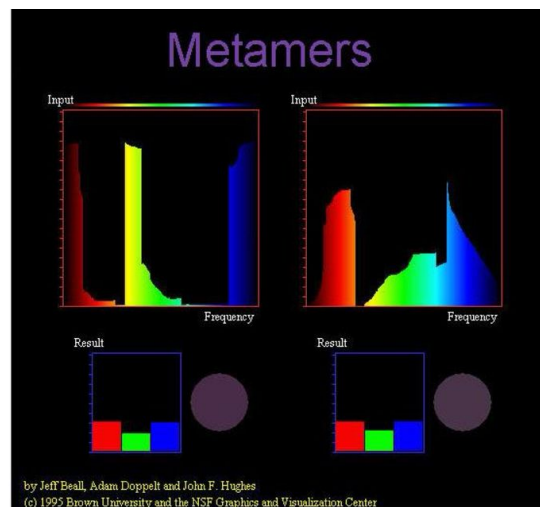
- **Rods** (≈ 120 million): extremely light-sensitive, active at night (scotopic vision), **cannot** encode color.
- **Cones** ($\approx 5\text{--}6$ million): less sensitive, active during the day (photopic vision), responsible for color vision. Three types, named by the peak wavelength they respond to.

Cone	Peak sensitivity	Perceived as
S (short)	≈ 440 nm	blue
M (medium)	≈ 530 nm	green
L (long)	≈ 560 nm	red

Tetrachromats are people with a 4th cone type, occasionally sensitive further into UV.

Metamers

Two physically different light spectra can trigger the **same** cone activations. The brain then perceives them as the identical color — it never has access to the raw spectrum, only to the three cone responses.



Different input spectra $\rightarrow$ same (R,G,B) cone activation $\rightarrow$ same perceived color.

2 Sensing Light & Color Filters

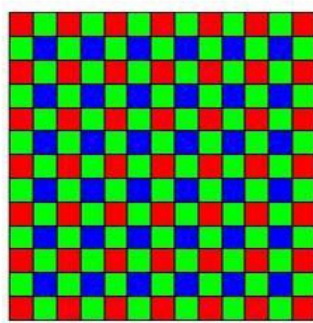
Camera sensor chips measure light **intensity**, not wavelength — color has to be reconstructed indirectly.

3-Chip vs. 1-Chip cameras

- **3-chip:** incoming light is split (via a prism) onto three separate sensors, one per color channel. Very accurate, but expensive and bulky.
- **1-chip:** a color filter mosaic sits directly on top of a single sensor array, so each photosite only receives one color.

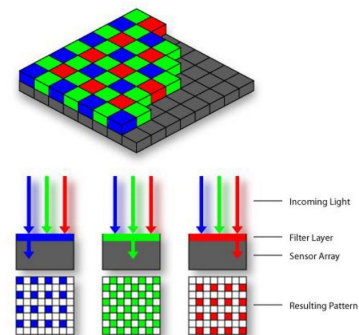
The Bayer Filter

The dominant 1-chip mosaic. Uses a **2:1:1** ratio of green : red : blue, because the human eye is most sensitive in the green part of the spectrum — mimicking this improves perceived sharpness and luminance resolution.



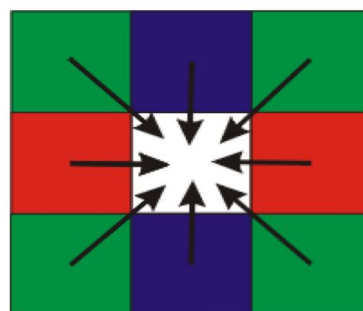
Bayer filter

Bayer mosaic pattern



Filter layer over the sensor array

Demosaicing: each pixel only has 1 of 3 color values recorded — the other two channels are *interpolated* from neighboring pixels.



Estimating full RGB per pixel from the surrounding mosaic values.

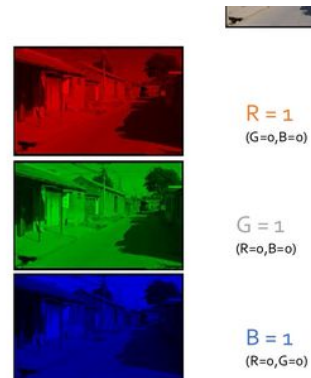
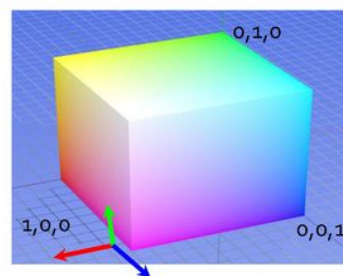
3 Color Spaces

RGB

Any color = $r \cdot R + g \cdot G + b \cdot B$. Simple and hardware-native, but:

- strongly **correlated** channels (shadows / highlights move all three together)
- **non-perceptual** — Euclidean distance in RGB space does not match perceived color difference

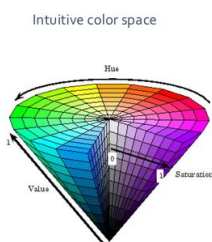
Any color = $r \cdot R + g \cdot G + b \cdot B$
 • Strongly correlated channels
 • Non-perceptual



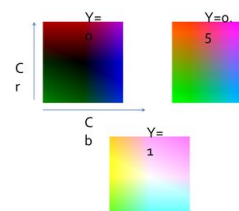
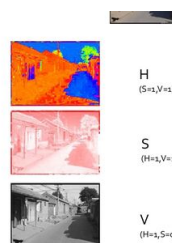
RGB cube and the three separated channel images of a scene.

HSV — Hue, Saturation, Value

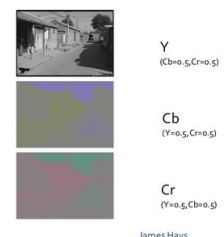
An intuitive, conical re-parametrization: **Hue** = color angle, **Saturation** = colorfulness, **Value** = brightness. More intuitive than RGB for e.g. color-based thresholding, but still not perceptually uniform.



HSV cone



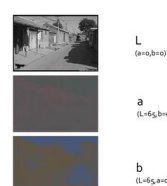
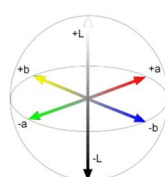
H / S / V channels of a photo



James Hays

YCbCr

Y = luma (brightness), C_b , C_r = blue-difference and red-difference chroma components. Separates brightness from color — exactly what JPEG and video codecs exploit (Section 7), since the eye is far less sensitive to chroma resolution than to luma resolution.



The C_r - C_b plane at different luma levels Y .

L*a*b*

Designed to be **perceptually uniform**: Euclidean distance between two points approximates the color difference actually perceived by humans. L = lightness, a = green↔red axis, b = blue↔yellow axis.

4 Image Filtering

2D Correlation vs. 2D Convolution

Correlation slides a filter f over the image I and does elementwise multiply-and-add:

$$h[m, n] = \sum_{k, l} f[k, l] I[m + k, n + l]$$

Convolution first flips the filter horizontally *and* vertically, then does the same sliding sum:

$$h[m, n] = \sum_{k, l} f[k, l] I[m - k, n - l] = (f * I)[m, n]$$

For a **symmetric** filter, 2D correlation and 2D convolution give identical results.

Example — flipping a filter for convolution

$$f_{\text{corr}} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}_{\text{origin top-left}} \longrightarrow f_{\text{conv}} = \begin{pmatrix} 4 & 3 \\ 2 & 1 \end{pmatrix}_{\text{origin bottom-right}}$$

Properties of convolution

Property	Rule	Note
Commutative	$a * b = b * a$	doesn't matter what is "filter" and what is "signal"
Associative	$a * (b * c) = (a * b) * c$	precompute a combined filter once, then convolve a single time. Correlation is not associative.
Distributive	$a * (b + c) = a * b + a * c$	over addition

Output size of a 1D convolution of a signal of length img with a kernel of length k : $\text{img} + k - 1$.

Box Filter

Simple smoothing / low-pass filter, e.g. a 3×3 box:

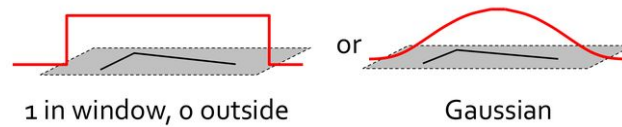
$$f_{\text{box}} = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Its frequency response is a sinc function $\mathcal{F}(f_{\text{box}})(\omega) = \frac{\sin \omega}{\omega}$, which has side-lobes → causes **ringing** artifacts.

Gaussian Filter

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Low-pass, and crucially its Fourier transform is *also* Gaussian — so **no ringing**. σ controls filter strength: bigger $\sigma \rightarrow$ more smoothing.



Box window vs. Gaussian window — the Gaussian avoids the hard cutoff that causes ringing.

Separability

A 2D kernel is separable if it factors into two 1D filters (an outer product of a column and a row vector):

$$K = v \cdot h^T$$

The 2D Gaussian is separable:

$$G_{\sigma}(x, y) = \left(\frac{1}{\sqrt{2\pi}\sigma} e^{-x^2/2\sigma^2} \right) \cdot \left(\frac{1}{\sqrt{2\pi}\sigma} e^{-y^2/2\sigma^2} \right)$$

Numerical example

$$\begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 2 & 1 \end{pmatrix}$$

Runtime — $N \times N$ image, $K \times K$ kernel

- No separability (full 2D convolution): K^2 ops/pixel $\rightarrow \mathcal{O}(N^2 K^2)$
- Separable (two 1D convolutions): $2K$ ops/pixel $\rightarrow \mathcal{O}(2N^2 K)$
- Speedup factor $K/2$ — e.g. a 9×9 kernel is $\approx 4.5\times$ faster separated.

Sobel Filter

Approximates the image gradient — a smoothing pass in one direction, a differencing pass in the other (and therefore separable):

$$K_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} \text{ (finds vertical edges)} \quad K_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix} \text{ (finds horizontal edges)}$$

5 Non-linear Filters & Morphology

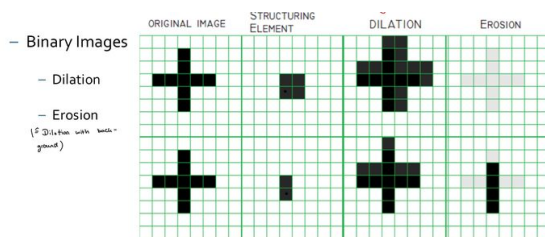
Median Filter

Non-linear — removes salt-and-pepper noise while keeping edges intact. Because it involves *sorting* pixel values, it cannot be expressed as a convolution.

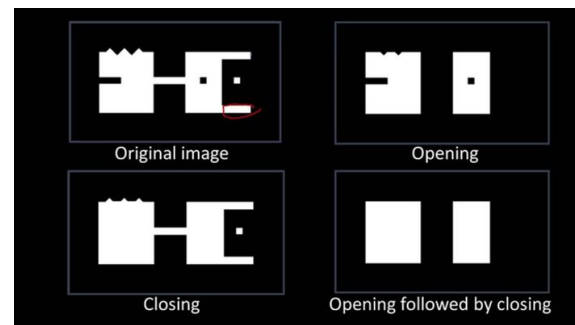
Morphological Operations

Change the shape of (binary) objects based on a **structuring element (SE)**. The SE must be mirrored before use, exactly as in convolution.

- **Erosion:** shrinks white areas — a pixel becomes white only if the SE fits *entirely* inside the white region.
- **Dilation:** grows white areas — a pixel becomes white if the SE merely *touches* the white region.
- **Opening** = Erosion → Dilation: removes small noise specks and thin protrusions.
- **Closing** = Dilation → Erosion: fills small holes and gaps.



Dilation vs. erosion on binary shapes



Opening / closing on an “H”-shaped object

6 Frequency Domain

Nyquist–Shannon Sampling Theorem

$$f_{\text{sample}} \geq 2 f_{\text{max}}$$

If violated → **aliasing**: high-frequency content gets distorted and produces artifacts (e.g. moiré patterns).

Downsampling — Image Pyramid

Repeat: blur with a Gaussian filter (low-pass, to satisfy Nyquist first), then subsample by a factor of 2. Each pyramid level halves the resolution.

Fourier Transform

Any univariate function can be rewritten as a weighted sum of sines and cosines of different frequencies (Euler: $e^{i\varphi} = \cos \varphi + i \sin \varphi$).

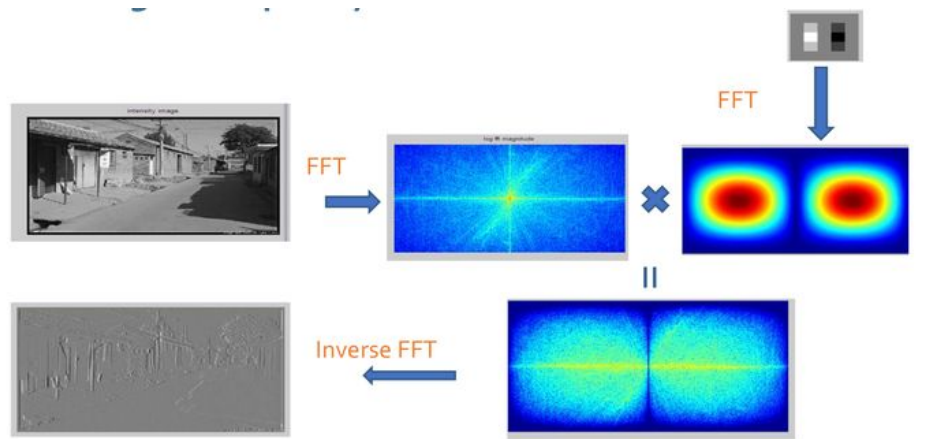
- **Amplitude:** how much signal is present at frequency f : $A = \sqrt{\text{Re}(f)^2 + \text{Im}(f)^2}$
- **Phase:** encodes spatial structure: $\Phi = \tan^{-1}(\frac{\text{Im}(f)}{\text{Re}(f)})$ — *most of the structural information of an image lives in the phase, not the amplitude.*

Convolution Theorem

$$\mathcal{F}[g * h] = \mathcal{F}[g] \cdot \mathcal{F}[h]$$

Convolution in the spatial domain $\equiv$ pointwise multiplication in the frequency domain.
Filtering pipeline:

1. FFT the image and the kernel
2. multiply elementwise in the frequency domain
3. inverse-FFT back to the spatial domain



Spatial-domain convolution $\equiv$ frequency-domain multiplication of the two FFTs.

Deconvolution

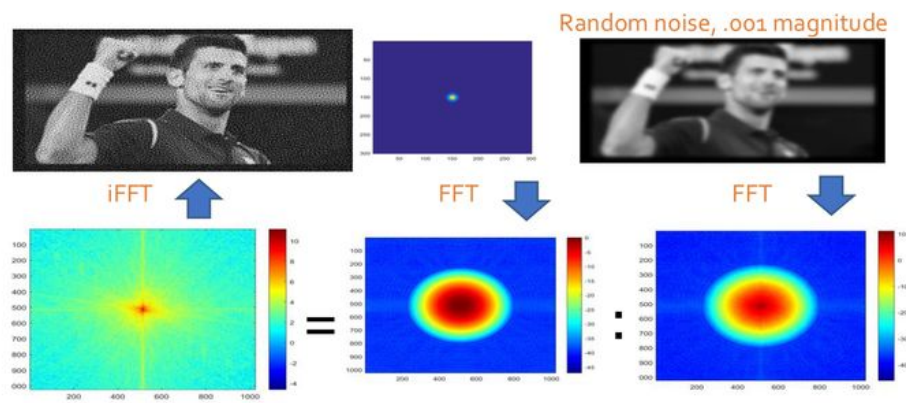
Numerically ill-posed. *Blind* (filter H unknown) vs. *non-blind* (filter known). Model $O = I * H$, so in the frequency domain $\hat{O}(\omega) = \hat{I}(\omega)\hat{H}(\omega)$, giving the naive inverse:

$$\hat{I}(\omega) = \frac{\hat{O}(\omega)}{\hat{H}(\omega)}$$

In reality there is noise N :

$$\hat{I}_{\text{real}}(\omega) = \hat{I}(\omega) + \frac{\hat{N}(\omega)}{\hat{H}(\omega)}$$

Since most real filters H are low-pass, at high frequencies $\hat{H}(\omega) \rightarrow 0$ — dividing a roughly constant noise floor by an ever-smaller number makes the result **explode** (and it is undefined where $\hat{H}(\omega) = 0$). Deconvolution therefore needs strong regularization.



Blurred/noisy image, its FFT, and the noise-dominated naive-deconvolution result.

7 JPEG Compression

1. Switch color space: RGB $\rightarrow$ YCbCr.
2. Subsample the chroma channels (C_b , C_r) by a factor of 2 — the eye perceives color at lower spatial resolution than luma.
3. Split the image into 8×8 patches; subtract 128 so values are roughly centered around 0.
4. For each block:
 - a) **DCT** (Discrete Cosine Transform) $\rightarrow$ frequency-domain block G .
 - b) **Quantize**: divide the DCT coefficients elementwise by a “magic” quantization matrix Q : $B = \lceil G/Q \rceil$. High frequencies get quantized to (near) zero — most perceptual information lives in the low frequencies, and the DC (top-left) coefficient is usually the largest.
 - c) **Zig-zag scan + Huffman encode**: the zig-zag ordering groups the now-mostly-zero high-frequency coefficients into long runs, which compress extremely well.

The decoder needs to know Q to reverse the compression — this is exactly the “quality” knob of a JPEG encoder.

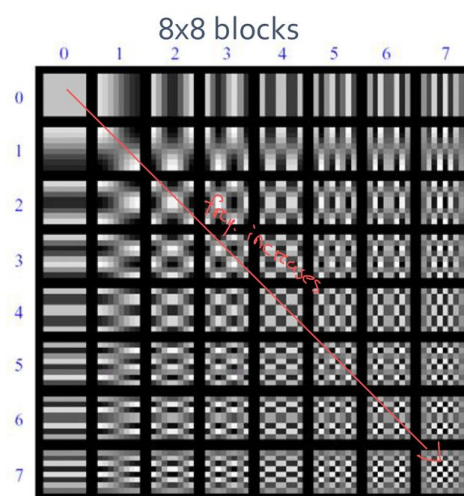


Image split into 8×8 blocks for the DCT / quantization pipeline.

8 Gamma Correction

Physics: in CRT monitors, output brightness I was not proportional to input voltage V :

$$I \propto V^\gamma, \quad \gamma \approx 2.2$$

To compensate, images are pre-corrected before display:

$$x_{\text{display}} = x_{\text{linear}}^{1/\gamma} = x_{\text{linear}}^{1/2.2}$$

Gamma correction is non-linear

If you run convolution / filtering directly on gamma-encoded (sRGB) pixel values, you are computing with the wrong physical quantity. Correct order of operations:

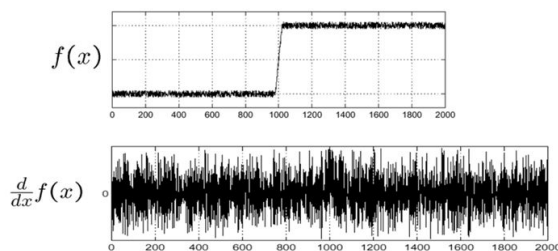
1. Undo gamma: $x_{\text{linear}} = x_{\text{sRGB}}^{2.2}$
2. Apply the actual filter / processing logic in linear space
3. Re-apply gamma for display and saving: $x_{\text{output}} = x_{\text{linear}}^{1/2.2}$

9 Edge Detection & Canny

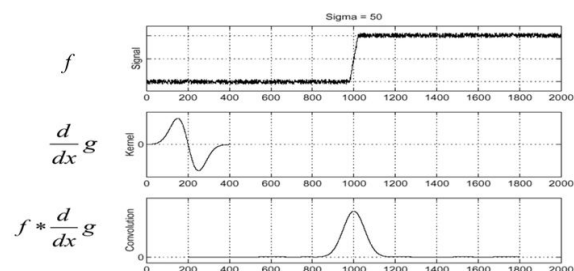
An edge is an **extremum in the derivative** of the 1D image intensity profile.

Problem: real line profiles are noisy, so the raw derivative signal is extremely spiky. Solution: filter (smooth) first, then differentiate — or better, precompute the derivative of the Gaussian kernel and convolve *once*:

$$\frac{d}{dx}(I * G) = I * \frac{d}{dx}G \quad (\text{saves one operation})$$



Step edge and its raw, noisy derivative



After Gaussian-derivative filtering — clean peak

An edge is a sudden change in one of four physical quantities

- **Geometry (normal):** a changing angle between surface normal N and light direction L → changing intensity (Lambertian: $I \propto N \cdot L$, with $\|N\| = 1$, N pointing outward).
- **Depth (occlusion):** a different surface behind → different lighting.
- **Material (albedo):** different reflection characteristics.
- **Light (shadow):** less incoming light.

Canny Edge Detector — the full pipeline

1. **Filter** the image with the x - and y -derivative of a Gaussian: $\frac{d}{dx}(I * G) = I * \frac{d}{dx}G$ — saves a step versus filtering and then differentiating.
2. **Gradient magnitude & orientation**: $M = \sqrt{I_x^2 + I_y^2}$, $\theta = \text{atan2}(I_x, I_y)$.
3. **Non-maximum suppression (NMS)**: keep a pixel only if it is larger than its neighbours *along the gradient direction*, interpolating between the two nearest pixels on that line (the gradient direction generally does not align with the pixel grid).
4. **Hysteresis thresholding** — avoids edges getting “cut” by noise. Uses two thresholds $t_{\text{high}}, t_{\text{low}}$:
 - $M > t_{\text{high}}$: strong edge (kept)
 - $M < t_{\text{low}}$: no edge (discarded as noise)
 - in between: weak edge — kept *only if connected* to a strong edge

Edge curves are grown starting from strong-edge pixels, extending through connected weak-edge pixels.

10 Corner Detection (Harris)

What makes a good feature?

- **Repeatability**: find the same feature again under geometric and photometric transformations.
- **Saliency**: each feature is distinctive.
- **Efficiency**: $\# \text{features} \ll \# \text{pixels}$.
- **Locality**: occupies a small region $\rightarrow$ robust to clutter and occlusion.

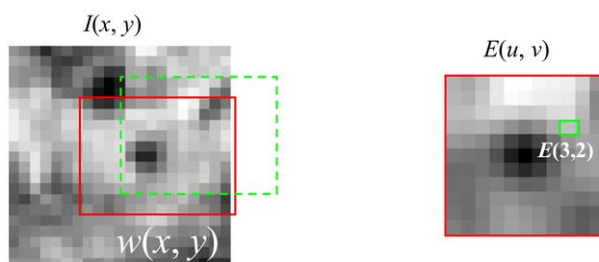
Corners satisfy all four criteria well.

Corner detection by auto-correlation

Look at a corner through a small window and shift it in any direction — a true corner produces a **large intensity change** in every shift direction:

$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

where $w(x, y)$ is a window function (a flat “1 inside / 0 outside” box, or a Gaussian). Computing this exactly for every pixel and every shift is expensive.



Auto-correlation surface $E(u, v)$ computed around a window.

Second-order Taylor approximation

Approximating $E(u, v)$ to second order around $(0, 0)$ — the linear term vanishes — gives:

$$E(u, v) \approx \begin{pmatrix} u & v \end{pmatrix} M \begin{pmatrix} u \\ v \end{pmatrix}, \quad M = \sum_{x,y} w(x, y) \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix}$$

M is the **second-moment matrix** or **structure tensor** — it locally captures “how pointy” the region is, built from products of the image gradients I_x, I_y .

Eigenvalues of M

The eigenvalues λ_1, λ_2 tell us about the local surface structure:

- both small $\rightarrow$ flat region, little structure
- one large, one small $\rightarrow$ edge
- both large ($\lambda_1 \approx \lambda_2$) $\rightarrow$ **corner**

Geometrically,

$$M = R^{-1} \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix} R$$

describes an ellipse: a big λ means a short axis and fast intensity change; a small λ means a long axis and slow change.

Cornerness response

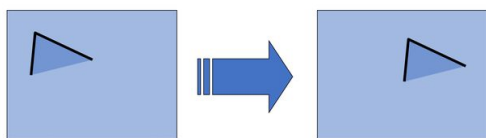
Computing eigenvalues per pixel is expensive, so use the trace and determinant of M instead:

$$C = \det(M) - \alpha \cdot (\text{trace}(M))^2$$

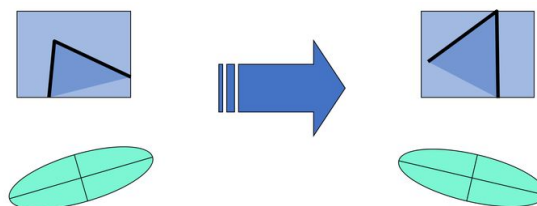
Decision: $C > \text{threshold}$ and large $\rightarrow$ corner; $|C|$ small $\rightarrow$ flat region; $C < 0 \rightarrow$ edge.

Invariance vs. covariance

Transformation	Behaviour
Translation	Covariant — corners shift together with the whole image.
Rotation	Covariant — the M -ellipse rotates, but its eigenvalues (hence C) stay the same.
Intensity shift $I \rightarrow I + b$	Invariant — only derivatives are used, so a constant offset cancels.
Intensity scaling $I \rightarrow aI$	Can cause problems — scaling changes the derivative magnitude.
Scaling / zoom	Not invariant or covariant — a corner at one scale can look like an edge at another.



Translation: covariant



Rotation: covariant

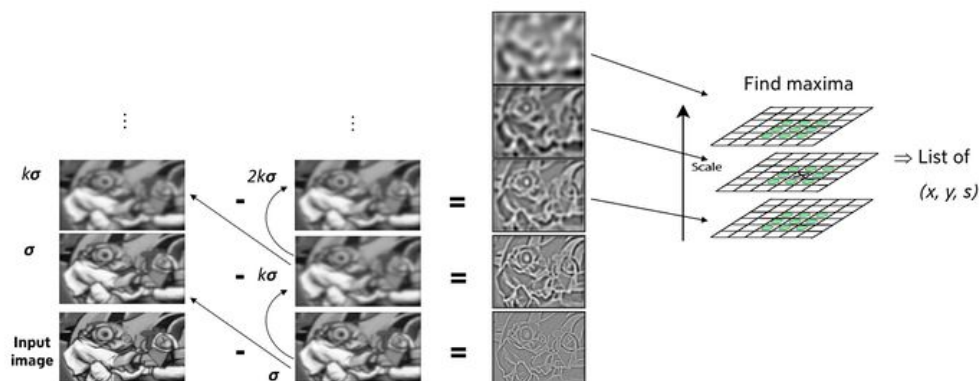
11 Features & Matching — SIFT

Goal: find **blobs** — a circular or oval region that differs substantially (in color or brightness) from its surroundings, e.g. a star in the night sky or the headlight of a car.

Step 1 — Find keypoints

A Laplacian-of-Gaussian (LoG) filter would find blobs directly but is expensive to compute, so SIFT approximates it with a **Difference of Gaussians (DoG)**:

- Filter the same image with a Gaussian at several different σ values, building a stack of increasingly blurred images.
- Subtract each pair of neighbouring blur levels $\rightarrow$ a stack of DoG “difference images” (different σ detects blobs of different radii / scales).
- Find local extrema by comparing each pixel to its **26 neighbours** (8 in-scale, 9 above, 9 below). A pixel becomes a **keypoint** if it is the local maximum *or* minimum among all 26.



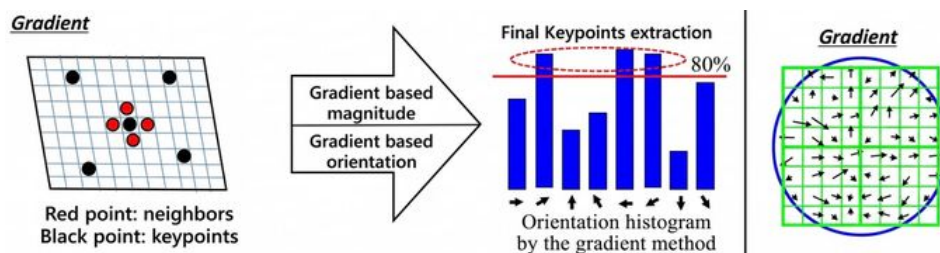
Gaussian scale-space stack $\rightarrow$ Difference-of-Gaussians $\rightarrow$ per-pixel 26-neighbour extremum search across scale.

Post-processing

Subpixel interpolation of the keypoint position; discard low-contrast points; discard points that lie along an edge or axis (same reasoning as Harris — an edge only constrains one direction).

Step 2 — Orientation assignment

Goal: make the descriptor rotation-invariant. Compute a dominant orientation per keypoint from a local **gradient-orientation histogram** (weighted by gradient magnitude), then normalize by rotating the patch to align with that dominant direction.

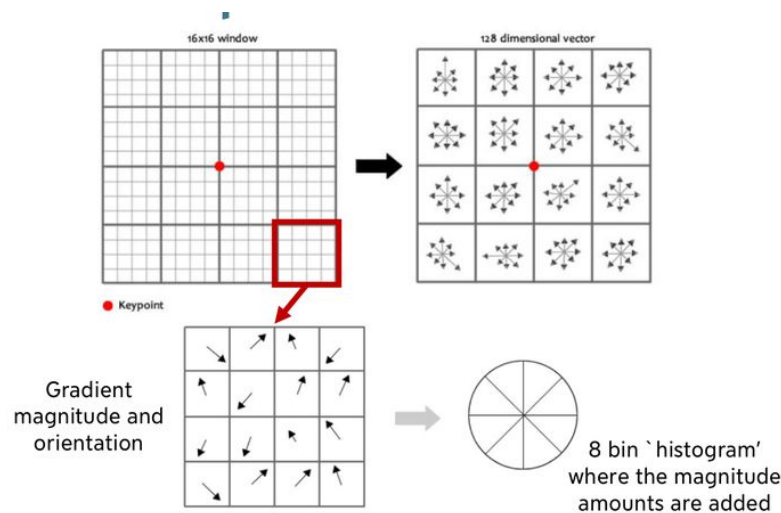


Gradient magnitude and orientation around a keypoint $\rightarrow$ orientation histogram $\rightarrow$ dominant orientation.

Step 3 — Descriptor extraction

Extract a 16×16 window around the (now normalized) keypoint and split it into 16 sub-blocks of 4×4 pixels. For each sub-block compute an 8-bin, magnitude-weighted orientation histogram:

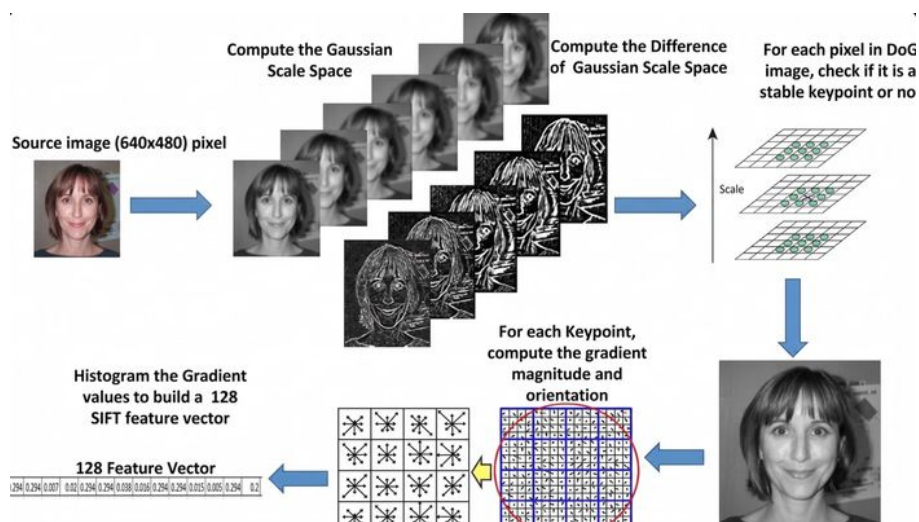
$$\underbrace{4 \times 4}_{\text{blocks}} \times \underbrace{8}_{\text{bins}} = 128\text{-dimensional feature descriptor}$$



16×16 patch $\rightarrow 4 \times 4$ grid of cells $\rightarrow$ per-cell 8-bin orientation histogram $\rightarrow$ 128-D descriptor.

Properties of SIFT features

- **Invariant to intensity shifts** ($I \rightarrow I + b$): only gradients are used, so a constant offset is removed.
- **Invariant to intensity scaling** ($I \rightarrow aI$): the final feature vector is normalized to unit L_2 -norm.



End-to-end: source image $\rightarrow$ Gaussian scale-space $\rightarrow$ DoG $\rightarrow$ stable keypoints $\rightarrow$ gradient histograms $\rightarrow$ 128-D SIFT feature vector.

Feature matching — Nearest Neighbour Distance Ratio (NNDR)

$$\text{ratio} = \frac{d(\text{Descriptor}, \text{NN}_1)}{d(\text{Descriptor}, \text{NN}_2)}$$

where NN_1 and NN_2 are the nearest and second-nearest descriptors in image 2 (Euclidean distance). A ratio near 0 means a confident, unambiguous match; a ratio near 1 means an ambiguous match that should be discarded. Choosing the right threshold needs labeled data, since the correct- and incorrect-match distributions overlap.

12 Cameras & Optics

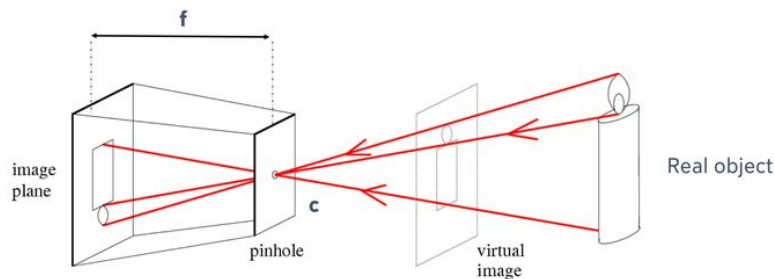
Pinhole Camera Model

By similar triangles, a 3D point at depth z projects to the image coordinate:

$$v = y \cdot \frac{f}{z}, \quad u = -x \cdot \frac{f}{z}$$

The minus sign appears because the real image on the sensor is flipped upside-down through the pinhole. Introducing a **virtual image plane** in front of the camera center at distance f removes it:

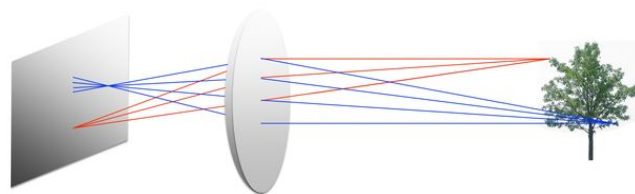
$$v = y \cdot \frac{f}{z}, \quad u = x \cdot \frac{f}{z}$$



Real (flipped) image behind the pinhole vs. the virtual image plane in front of it.

Underlying physics

- Bigger focal length $f \rightarrow$ the image appears bigger (zoom).
- Bigger pinhole $\rightarrow$ more light ($\uparrow$ SNR) but $\downarrow$ sharpness; smaller pinhole $\rightarrow$ $\downarrow$ SNR but $\uparrow$ sharpness. Real cameras need **both**, hence lenses.
- Lenses can only be focused for one specific object distance at a time $\rightarrow$ objects at other distances appear defocused (blur circle).



A lens focuses light from one distance sharply onto the sensor; other distances defocus.

13 Homogeneous Coordinates, Points & Lines

Translation $x_{\text{trans}} = x + t$ cannot be written as a single matrix multiplication in plain Cartesian coordinates, so we switch to **homogeneous coordinates**, which turn every affine map into one matrix multiply.

	Cartesian	Homogeneous
2D	(x, y)	$(x, y, 1)^T$
3D	(x, y, z)	$(x, y, z, 1)^T$

Homogeneous $\rightarrow$ Cartesian: divide by the last component w , then drop it:

$$(x, y, w)^T \rightarrow \left(\frac{x}{w}, \frac{y}{w} \right)$$

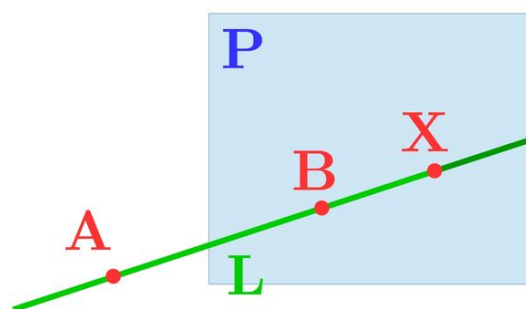
Scale invariance in projective space: $k \cdot (x, y, w)^T$ represents the *same* point as $(x, y, w)^T$ for any $k \neq 0$.

Points & lines (2D)

A line $ax + by + c = 0$ corresponds to the homogeneous vector $\ell = (a, b, c)^T$; a point corresponds to $p = (u, v, 1)^T$. A point lies on a line iff $\ell \cdot p = 0$ (dot product zero, i.e. orthogonal in the homogeneous sense).

Operation	Formula
Join — line through two points a, b	$\ell \cong a \times b$
Meet — intersection of two lines ℓ, m	$a \cong \ell \times m$

Parallel lines have the same (a, b) but different c , so the third component of their cross product is always zero. Two parallel lines therefore meet at $w = 0$ — a **point at infinity**.



Join / meet, and the point at infinity where parallel lines “intersect”.

14 The Camera Matrix

Using homogeneous coordinates, the full $3D \rightarrow 2D$ camera projection becomes a single matrix multiplication:

$$w \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K [R \mid t] \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}, \quad w = \text{depth } Z \text{ in the camera frame}$$

Intrinsic matrix K — 5 DoF

$$K = \begin{pmatrix} f_x & s & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{pmatrix}$$

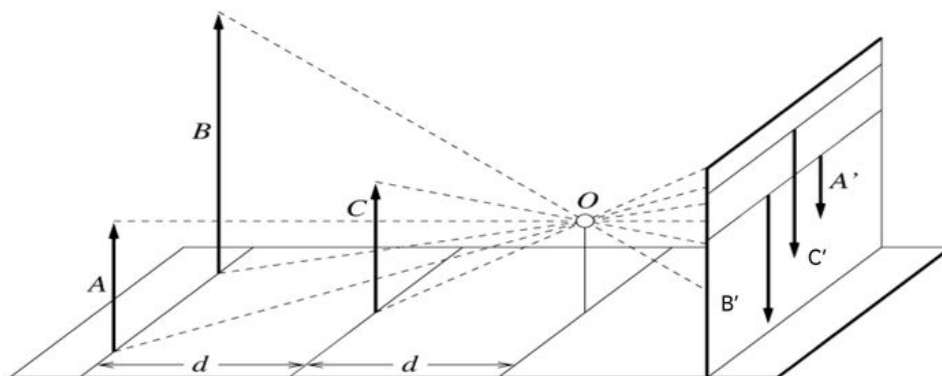
Parameter	Meaning	Visual effect
f_x, f_y	focal length in pixels, $f_x = f \cdot \frac{1}{\text{px width}}$	bigger $f_x \rightarrow$ image appears bigger (zoom)
$f_x \neq f_y$	non-square pixels	image gets stretched or squeezed in one direction
s (skew)	pixel axes not orthogonal	image looks sheared; in practice $s \approx 0$ almost always
u_0, v_0	principal point — pixel coordinate where the optical axis hits the sensor	shifts the entire image left/right or up/down; changes with pinhole shift

Extrinsic matrix $[R \mid t]$ — 3 DoF rotation + 3 DoF translation

Describes the camera's position in the world: R = rotation matrix, t = translation. An important subtlety — the camera center C is **not** t :

$$C \neq t, \quad C = -R^{-1}t = -R^T t$$

Total DoF of the camera matrix $M = K[R \mid t]$: **11**.



World coordinate frame vs. camera coordinate frame — the camera can be positioned and rotated anywhere in the world.

15 Singular Value Decomposition (SVD)

$$A = U \Sigma V^T$$

- Σ : diagonal matrix of singular values $\sigma_i \geq 0$, sorted descending.
- U : orthogonal matrix whose columns are the *left* singular vectors, $U^T U = I$.
- V : orthogonal matrix whose columns are the *right* singular vectors, $V^T V = I$.

Geometric picture: $A = U \Sigma V^T$ decomposes any linear map into **rotate** $\rightarrow$ **stretch along axes** $\rightarrow$ **rotate**.

Solving least-squares systems with SVD

Problem	Solution
$Ax = b$, minimize $\ Ax - b\ ^2$	$x = V \Sigma^{-1} U^T b$ (pseudo-inverse of A), or <code>np.linalg.lstsq(A, b)</code>
$Ax = 0$ (homogeneous / total least squares)	<code>U, S, Vh = np.linalg.svd(A)</code> ; the optimal x is the last row of V^h (= last column of V), i.e. the singular vector belonging to the <i>smallest</i> singular value: <code>x = Vh[-1, :]</code>

Forcing rank-deficiency

Used e.g. to reduce the rank of the fundamental matrix F from 3 to 2 (Section 21):

1. Decompose $F = U \cdot \text{diag}(\sigma_1, \sigma_2, \sigma_3) \cdot V^T$
2. Set the smallest singular value to zero: $\Sigma_{\text{new}} = \text{diag}(\sigma_1, \sigma_2, 0)$
3. Recombine: $F_{\text{corr}} = U \Sigma_{\text{new}} V^T$

Aligning 3D scans: given two point clouds $\{p_i\}, \{q_i\}$, find the rigid transform (rotation R + translation t) that best aligns them — use SVD to find the optimal rotation matrix (Procrustes, Section 24).

16 2D Transformations

Linear transformations map through a matrix — scaling, rotation, mirroring, shear:

$$\text{scale: } \begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix} \quad \text{rotate: } \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

Properties: the origin stays fixed, lines map to lines, parallel lines stay parallel, ratios along a line are preserved, and the set is closed under composition.

Problem: translation is not linear in plain 2D coordinates. Using homogeneous coordinates makes it a matrix multiply too, at which point the classic transform hierarchy appears:

Name	Matrix	DoF	Invariances
Translation	$[I \mid t]_{2 \times 3}$	2	orientation, lengths, angles, parallelism
Rigid (Euclidean)	$[R \mid t]_{2 \times 3}$	3	lengths, angles, parallelism
Similarity	$[sR \mid t]_{2 \times 3}$	4	angles / shapes, parallelism, ratios
Affine	$[A]_{2 \times 3}$	6	parallelism, straight lines, length ratios along parallel lines
Projective	$[H]_{3 \times 3}$	8	only straight lines stay lines; parallelism is lost; defined up to scale

17 Camera Calibration (Direct Linear Transform)

Given 3D points X_i and their known 2D image correspondences (u_i, v_i) , recover the unknown camera matrix M (up to scale, 11 DoF):

$$w \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}$$

Each 3D–2D correspondence gives 2 linear equations after eliminating w , so with 11 DoF **6 points are the theoretical minimum**. In practice use far more for stability.

Two ways to solve

Formulation	Approach
$Ax = b$	Fix $m_{34} = 1$, move all m_{34} terms to the right-hand side, solve ordinary linear least squares: $x = (A^T A)^{-1} A^T b$, or <code>np.linalg.lstsq(A, b)</code> . Then append $m_{34} = 1$ and reshape to 3×4 .
$Ax = 0$	Total least squares via SVD: $U, S, V^h = \text{np.linalg.svd}(A)$; take the last row of V^h (smallest singular value) and reshape to 3×4 .

Decomposing M back into K, R, t

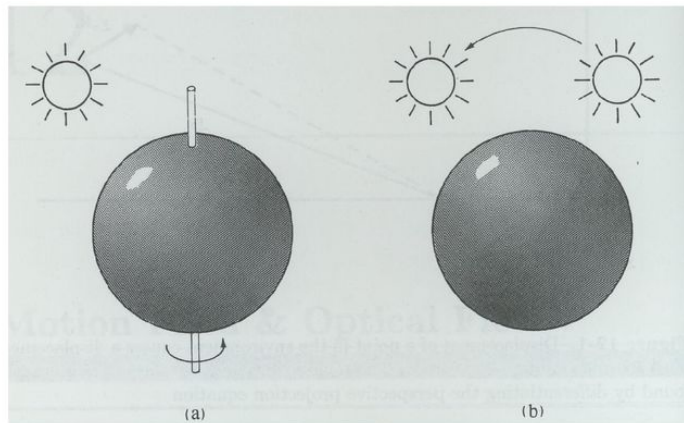
$M = K[R \mid t]$: the left 3×3 block KR is (upper-triangular) $\times$ (orthogonal), recovered via **RQ-decomposition**, which yields K and R . Then $t = K^{-1}M_{:,4}$.

18 Dense Motion Estimation

Optical flow $\neq$ motion flow

Optical flow is the apparent movement of *brightness patterns* in an image sequence; **motion flow** is the actual physical motion. They usually correlate, but not always:

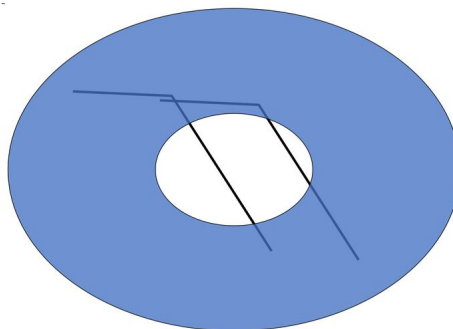
- A uniformly-lit sphere rotating in place $\rightarrow$ motion flow, but **no** optical flow (the brightness pattern does not change).
- A static sphere with a moving light source $\rightarrow$ the specular highlight moves $\rightarrow$ optical flow, but **no** motion flow.



Rotating sphere (motion, no optical flow) vs. moving highlight (optical flow, no motion).

The Aperture Problem

Looking at a straight edge through a small aperture, only the motion component *perpendicular* to the edge can be measured — motion *along* the edge is invisible.

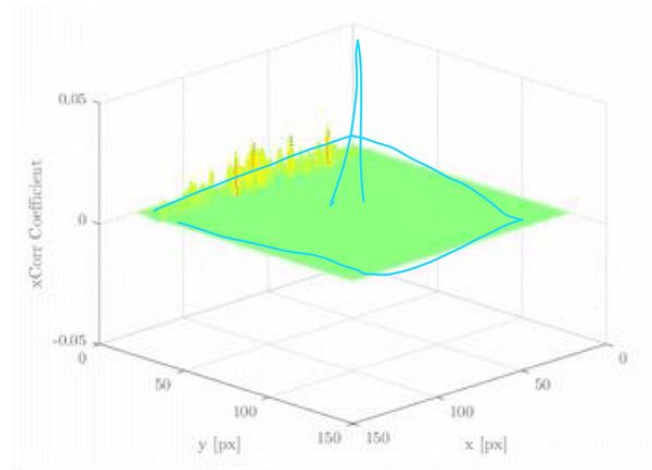


Only the motion component normal to the edge is observable.

Error metrics

Given $I_0(x)$, $I_1(x)$ with $x = (x, y)$, the goal is to find the translation $u = (u, v)$ that best aligns them.

Metric	Formula	Note
SSD	$\sum_i [I_1(x_i+u) - I_0(x_i)]^2$	large errors dominate
SAD	$\sum_i I_1(x_i+u) - I_0(x_i) $	large errors don't dominate as much
L_p -norm	$(\sum_i I_1(x_i+u) - I_0(x_i) ^p)^{1/p}$	generalizes SSD ($p=2$) and SAD ($p=1$)
Cross-correlation	$\sum_i I_0(x_i) I_1(x_i+u)$	strong, but not normalized
Normalized CC	centers by the mean, divides by std. dev.	clamps to $[-1, 1]$; brightness and contrast differences do not matter
Bias / gain	$\sum_i (I_1(x_i+u) - \alpha I_0(x_i) - \beta)^2$	needed when switching between cameras with different response curves



A sharp, well-defined correlation peak (good) vs. a flat, ambiguous surface (bad, low-texture region).

19 Lucas–Kanade Optical Flow

The go-to method to estimate optical flow efficiently, since pixel-wise search is far too expensive.

Basic problem

Given I_0 at t , I_1 at $t+1$, and a rough estimate $u = (u, v)^T$, search for a small sub-pixel correction Δu . Since Δu is small, a first-order Taylor approximation is valid:

$$I_1(x_i+u+\Delta u) \approx I_1(x_i+u) + J_1(x_i+u) \Delta u$$

where the Jacobian of I_1 is simply the local image gradient: $J_1(x_i+u) = \nabla I_1 = \begin{pmatrix} I_x & I_y \end{pmatrix}$.

Patch-wise SSD

Assumption: every pixel in an $n \times n$ patch shifts by the *same* Δu (local consistency):

$$\begin{aligned}
 E(\Delta u) &= \sum_{i=1}^n [I_1(x_i+u+\Delta u) - I_0(x_i)]^2 \\
 &\approx \sum_{i=1}^n [J_1(x_i+u) \Delta u + e_i]^2, \quad e_i = I_1(x_i+u) - I_0(x_i)
 \end{aligned}$$

This is exactly a linear least-squares problem $\|A\Delta u - b\|^2$ with:

$$A = \begin{pmatrix} I_x(x_1) & I_y(x_1) \\ \vdots & \vdots \\ I_x(x_n) & I_y(x_n) \end{pmatrix}_{(n \times 2)}, \quad b = \begin{pmatrix} e_1 \\ \vdots \\ e_n \end{pmatrix}_{(n \times 1)}$$

Minimization

$$\frac{d}{d\Delta u} \|A\Delta u - b\|^2 = 0 \implies A^T A \Delta u = A^T b$$

$$A^T A = \begin{pmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{pmatrix} \quad A^T b = \begin{pmatrix} \sum I_x (I_0 - I_1) \\ \sum I_y (I_0 - I_1) \end{pmatrix}$$

Note that $A^T A$ is exactly the same 2×2 **structure tensor** M as in the Harris corner detector (Section 10).

$A^T A$ has to be invertible. Lucas–Kanade fails in flat, low-texture regions, because there the structure tensor becomes singular ($\det = 0$).

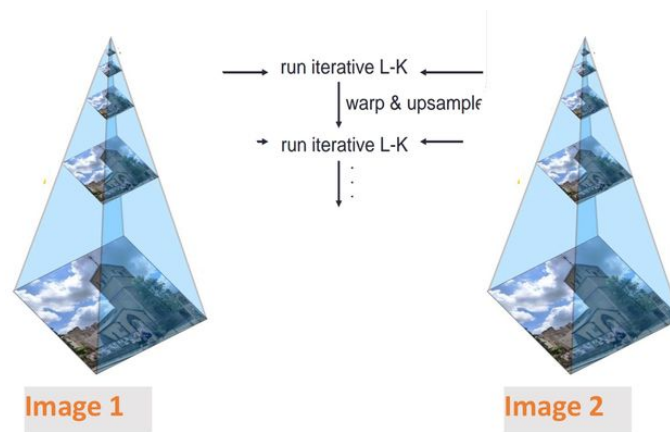
Iterative solver

```
u = [0, 0]
repeat until converged:
    A = gradients of I1 at x + u
    b = residuum (I0 - I1 at x+u)
    du = np.linalg.solve(A.T @ A, A.T @ b)
    u = u + du
```

Iteration is needed because the Taylor expansion is only valid for small Δu .

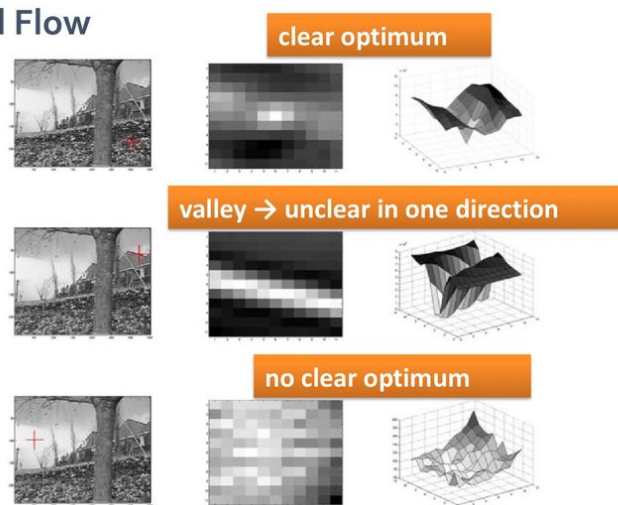
Coarse-to-fine (pyramidal LK)

Works well in high-texture regions; fails in flat regions or along repetitive structures (edges). To handle large motions, run it on an image pyramid: start at the coarsest level with $u = 0$, run iterative LK, then **upsample** u (by $\times 2$, since we move one level down the pyramid) and **warp** image 2 by the current estimate before refining at the next finer level. The residual error shrinks as the estimate improves.



Coarse-to-fine: estimate flow at a coarse pyramid level, upsample and warp, refine at the next finer level.

Flow



Patch-based flow: a clear single optimum (good), an unclear valley (ambiguous in one direction), and no clear optimum at all (flat/textured). Regularizing with a smoothness term $\lambda \|\nabla u\|^2$ helps resolve the ambiguous cases.

20 Epipolar Geometry

Projective geometry in 3D

Duality: in 2D, points $\leftrightarrow$ lines; in 3D, points $\leftrightarrow$ **planes**, and lines are self-dual.

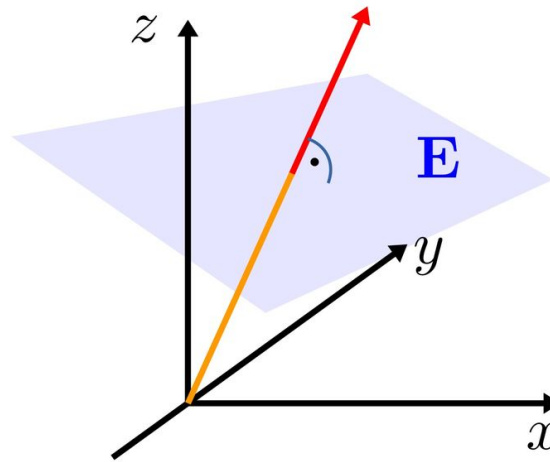
A 3D point is $A \cong (A_0, A_1, A_2, A_3)^T \in \mathbb{P}^3$; a 3D plane is $P \cong (P_0, P_1, P_2, P_3)^T \in \mathbb{P}^3$. A point $X = (x, y, z, 1)$ lies on plane $E = (e_0, e_1, e_2, e_3)$ iff the homogeneous scalar product vanishes:

$$X^T E = e_0 x + e_1 y + e_2 z + e_3 = 0$$

Hessian normal form of a plane

$$E = \begin{pmatrix} e_0 \\ e_1 \\ e_2 \\ e_3 \end{pmatrix} \cong \begin{pmatrix} n \\ -t \end{pmatrix} = \frac{1}{\sqrt{e_0^2 + e_1^2 + e_2^2}} \begin{pmatrix} e_0 \\ e_1 \\ e_2 \\ e_3 \end{pmatrix}$$

n is the unit normal vector of the plane; t is the signed distance from the origin, i.e. how far the plane is shifted along n .



Plane E with unit normal n and signed origin-distance t .

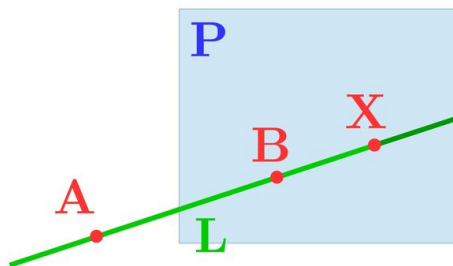
3D lines

A 3D line cannot be written as a simple 4-vector (that would be a plane) — it is the intersection of two planes, or the connection between two points. It is represented by the **Plücker matrix**, built from two points A, B on the line:

$$[L]_{\times} = \begin{pmatrix} 0 & -L_{01} & -L_{02} & -L_{03} \\ L_{01} & 0 & -L_{12} & -L_{13} \\ L_{02} & L_{12} & 0 & -L_{23} \\ L_{03} & L_{13} & L_{23} & 0 \end{pmatrix} = A B^T - B A^T$$

From it:

- **Line direction:** $d = (-L_{01}, -L_{13}, -L_{23})^T$ — where does it point in space?
- **Line moment:** $m = (L_{23}, -L_{02}, L_{01})^T$ — how is it shifted in space? With $d^T m = 0$ (orthogonal).
- **Distance from origin:** $d_{\text{orig}} = \|m\| / \|d\|$
- **Line $\cap$ plane:** $X = [L]_{\times} P$

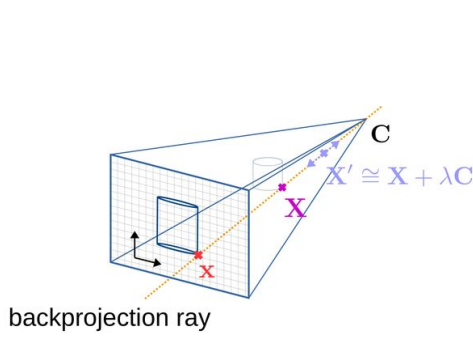


A 3D line L through points A, B , intersecting plane P at X .

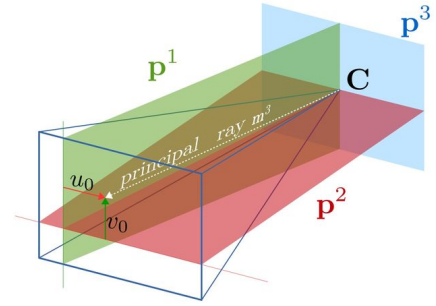
The projection matrix P

Intuition: projecting a 3D point X with camera matrix P means drawing a line from X to the camera center C and finding where that line intersects the image plane. Since $PC = 0$, **every** point on the same back-projection ray $X' = X + \lambda C$ projects to the exact same image point:

$$PX' = P(X + \lambda C) = PX + \lambda \underbrace{PC}_{=0} = PX$$

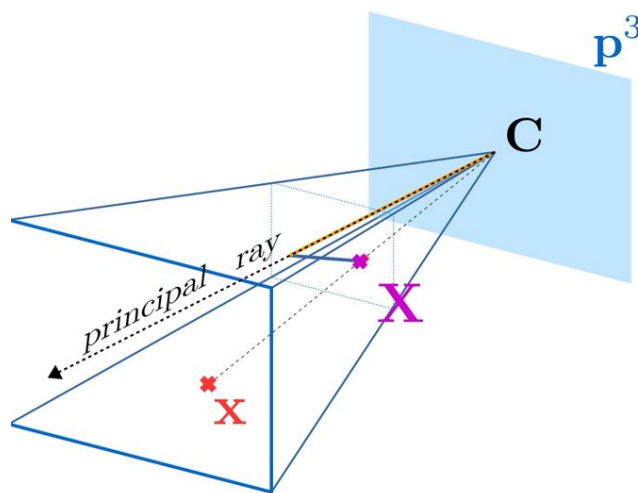


Back-projection ray: all points on it map to the same image point



The three principal planes p^1, p^2, p^3 of P

- **Back-projection ray** of an image point x : $R = \text{join}(P^+x, C)$, where P^+ is the pseudo-inverse of P (giving one point somewhere on the ray).
- **Back-projection plane** of an image line ℓ : $E = P^T \ell$. This plane always passes through the camera center, since $E^T C = (P^T \ell)^T C = \ell^T (PC) = 0$.
- **Principal plane** p^3 (the third row of P): parallel to the image plane and through the camera center. Everything on it maps to $w = 0$ in the image, i.e. to infinity — nothing on it can be represented on the sensor.
- **Principal ray** m^3 : the normal of the principal plane. Where it hits the image plane defines the **principal point** (u_0, v_0) .



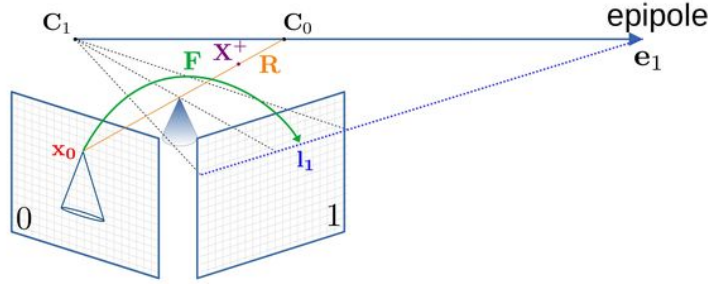
The principal ray hits the image plane at the principal point.

Depth of a 3D point is its perpendicular distance from the principal plane p^3 , provided $\|m^3\| = 1$ and $\det(M) > 0$.

Two-view geometry

The same 3D point X projects as $x_0 = P_0 X$ in camera 0 and $x_1 = P_1 X$ in camera 1. All rays through a fixed image point x_0 map, in image 1, to a single line ℓ_1 — the **epipolar line**. The projection of camera 0's own center into image 1 is the **epipole**:

$$e_1 \cong P_1 C_0$$



Baseline, epipoles, and the epipolar line ℓ_1 that a single point x_0 constrains matches to lie on.

21 The Fundamental Matrix

Deriving ℓ_1 from x_0 and the two projection matrices:

$$\ell_1 \cong e_1 \times x^+ = P_1 C_0 \times P_1 P_0^+ x_0 = \underbrace{([P_1 C_0] \times P_1 P_0^+)}_F x_0$$

F — the **fundamental matrix** — maps points x_0 in image 0 to their epipolar *lines* ℓ_1 in image 1.

Epipolar constraint

$$x_1^T F x_0 = 0$$

This must hold for **every** pair of corresponding points (x_0, x_1) — it is the single equation the whole two-view geometry rests on. The epipoles are the null-space of F : $F e_0 = 0$.

Estimating F — the 8-point algorithm

Expanding $x_1^T F x_0 = 0$ for each correspondence (found e.g. via SIFT) gives one linear equation in the 9 unknown entries of F :

$$u_1 u_0 f_{11} + u_1 v_0 f_{12} + u_1 f_{13} + v_1 u_0 f_{21} + v_1 v_0 f_{22} + v_1 f_{23} + u_0 f_{31} + v_0 f_{32} + f_{33} = 0$$

Stacking one row per correspondence gives $A \vec{f} = 0$.

1. **Normalize** (per image): shift the point cloud to a scaled centroid and rescale so the average distance from the origin is $\sqrt{2}$ — this dramatically improves numerical stability. Store the transforms T_0, T_1 and apply as $\hat{x} = T x$.
2. **Build A** — one row per correspondence:

$$[u_1 u_0, u_1 v_0, u_1, v_1 u_0, v_1 v_0, v_1, u_0, v_0, 1]$$

3. **Solve** $Af = 0$ via SVD: $A = U\Sigma V^T$, and f is the last column of V (last row of V^T) — the singular vector for the smallest singular value. Reshape f into the 3×3 matrix $\hat{F}$.
4. **Force rank 2:** F must be singular (rank-deficient — this is what guarantees all epipolar lines meet in a single epipole). SVD $\hat{F} = U \text{diag}(\sigma_1, \sigma_2, \sigma_3) V^T$, set $\sigma_3 = 0$, rebuild:

$$\hat{F}_{\text{corr}} = U \text{diag}(\sigma_1, \sigma_2, 0) V^T$$

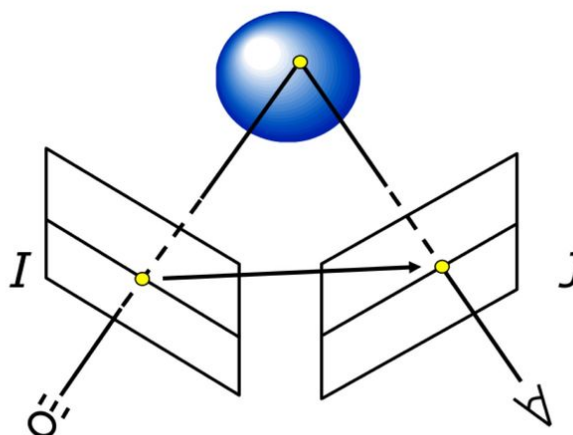
5. **Denormalize:** $F_{\text{corr}} = T_1^T \hat{F}_{\text{corr}} T_0$

22 Stereo Vision

Triangulation

For a point $Q = (x, y, z)$ seen by a left/right camera pair separated by baseline T , similar triangles on each camera give $x_L = x \frac{f}{z}$ and $x_R = (x - T) \frac{f}{z}$, which combine to:

$$z = f \cdot \frac{T}{x_L - x_R} = f \cdot \frac{T}{d}, \quad d = \text{disparity}$$



Depth from disparity via similar triangles.

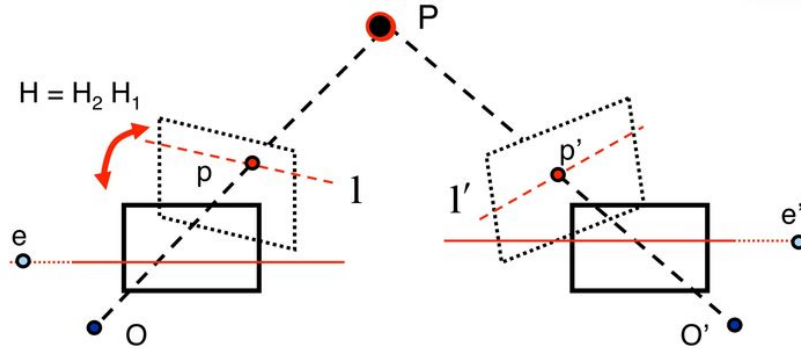
Impact of the baseline

Small baseline	Large baseline
+ similar images → few occlusions, easy correspondence search – large depth error, sensitive to noise (poor disparity resolution)	+ high depth precision, stable against sensor noise – hard search problem (occlusion, views look very different)

Solution — Vergence: turn the cameras toward each other. This gains a bigger shared field of view and better correspondence, but epipolar lines are no longer parallel (finite epipoles) and there is no scan-line coherence — a point is not in the same image row in both views.

Rectification

Problem: the simple disparity formula needs parallel optical axes, i.e. parallel epipolar lines that coincide with scan lines. **Solution:** warp both images with homographies H_0, H_1 that push both epipoles to infinity. Afterwards the epipolar lines are parallel and horizontal, turning stereo correspondence into a simple 1D search along scanlines.



Rectification: warp both images so the epipoles go to infinity and epipolar lines become parallel.

Window-based matching

Corresponding points lie on the same scanline (= epipolar line after rectification), but pixel-wise search alone fails due to sensor and quantization noise plus ambiguity (many pixels share similar intensity). Instead, compare small **patches**: slide a reference patch from I_0 across the corresponding scanline in I_1 .

Window size: small $\rightarrow$ more detail but more noise sensitivity; large $\rightarrow$ less noise but less detail (over-smoothed disparity).

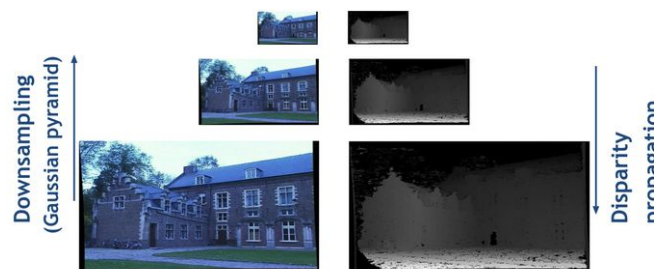
$$E_{SSD} = \sum_{[i,j] \in R} (f(i,j) - g(i+d, j))^2$$

$$C_{fg} = \sum_{[i,j] \in R} f(i,j) g(i+d, j)$$

$$NCC_{fg} = \frac{1}{n-1} \sum_{[i,j] \in R} \frac{(f(i,j) - \bar{f})(g(i+d, j) - \bar{g})}{\sigma_f \sigma_g}$$

Low-texture regions give high ambiguity — many disparities d score almost equally well — whereas high-texture regions give a sharp, unambiguous peak.

Better: image-pyramid based matching. Do coarse matching at higher (smaller) pyramid levels, then propagate the disparity result down and fine-tune with small windows at the original resolution.



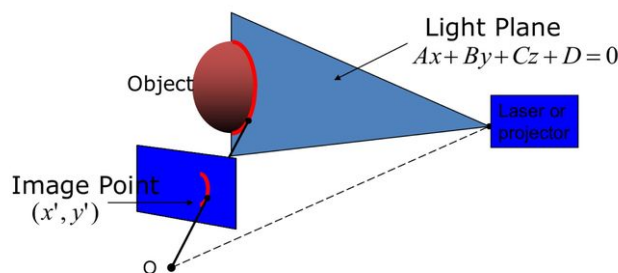
Gaussian downsampling pyramid with disparity propagation from coarse to fine resolution.

23 Structured Light & Time of Flight

Passive vs. active stereo

Passive stereo (Section 22): take two images and search for correspondences — a hard search problem that can fail in low-texture regions.

Active: replace one camera with a laser or projector. For each projector pixel: turn it on, search for the resulting bright dot in the camera, intersect the two rays → a 3D point. Slow (one point at a time) but very robust.



A projected light plane intersecting the object, imaged by a camera.

Triangulation with a light plane

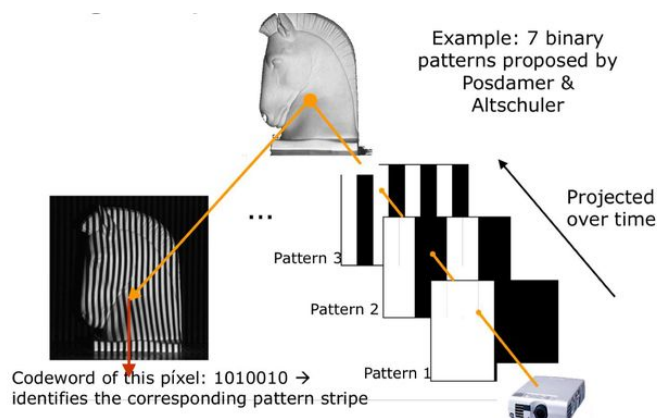
The projected plane $Ax + By + Cz + D = 0$ combined with the pinhole relations $x = x'z/f$ and $y = y'z/f$ gives:

$$z = \frac{-Df}{Ax' + By' + Cf}$$

Very precise, but slow, since the light plane has to be swept over the entire object.

Binary coded light striping

Much faster: project a sequence of binary striped patterns onto the object, one after another. Each point on the image plane then observes a unique binary code (bright = 1, dark = 0) over time — $(2^n - 1)$ distinguishable stripes for n images. Map the binary code to a light-plane equation $Ax + By + Cz + D = 0$, then triangulate z exactly as above.

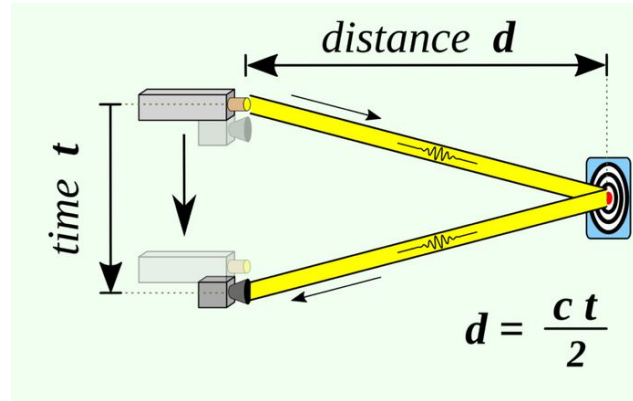


Sequential binary striping — the on/off sequence at one pixel encodes which projector column illuminated it.

Time of Flight (ToF)

Emit a short light pulse; the sensor measures the round-trip time t until the reflected light returns:

$$d = \frac{c \cdot t}{2}, \quad c \approx 2.998 \times 10^8 \frac{\text{m}}{\text{s}}$$



Time-of-flight distance measurement.

24 Surface Reconstruction

Task

Given a dense set of 3D points (from multiple scans, depth images, or stereo reconstruction), reconstruct a closed surface from these points.

Coarse alignment — Procrustes

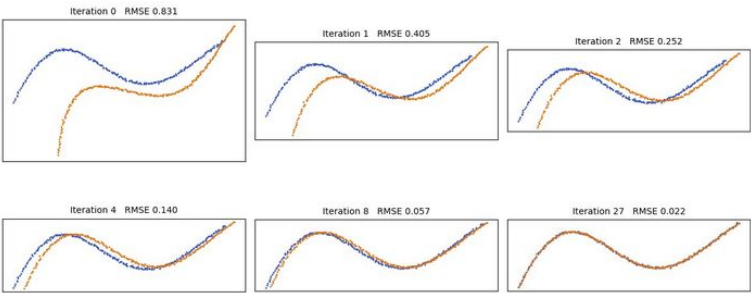
Given $\{p_i\}, \{q_i\}$, find rotation R and translation t :

$$\min_{R,t} \sum_i \|Rp_i + t - q_i\|^2 \quad \Rightarrow \quad \text{solve with SVD}$$

Fine alignment — Iterative Closest Point (ICP)

Iterate until converged:

1. Select points.
2. Find closest points (p_i, q_i) on the target (speed this up with a **KD-tree**).
3. Weight the correspondences c_i .
4. Prune outliers and low-confidence points ($c_i < t$).
5. Solve Procrustes on the remaining pairs to get R, t : $\min_{R,t} \sum_i \|p_i - (Rq_i + t)\|^2$ (point-to-point).
6. Transform the source data: $d' = d \cdot R + t$.

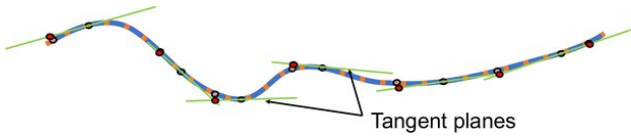


Alignment improving over successive ICP iterations, with the RMSE decreasing.

Point-to-point vs. point-to-plane

	Point-to-point	Point-to-plane
Measures	Euclidean distance between points	distance from a point to the <i>tangent plane</i> of the target point
Needs	—	point normal vectors (constructed via <i>k</i> -NN)
Trade-off	+ simple – slow to converge	+ convergence independent of point sampling, faster

Normal sampling of points works better than uniform or random sampling for both variants.



Point-to-plane alignment via local tangent planes.

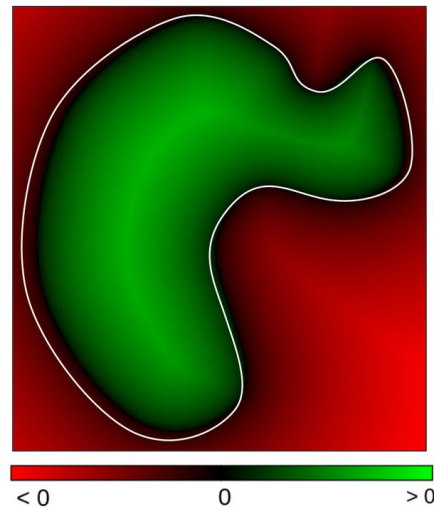
Outliers: since ICP works with SSD, outliers contribute hugely to the error — prune them.
Correspondence: only points in the overlapping region between the two scans have meaningful correspondences.

Implicit functions

Once the scans are aligned in one coordinate frame, we need to turn the point cloud into an actual surface — either a triangle mesh directly, or, better, an **implicit function**:

$$f : \mathbb{R}^3 \rightarrow \mathbb{R}, \quad \begin{cases} f(p) > 0 & \text{inside the object} \\ f(p) < 0 & \text{outside} \\ f(p) = 0 & \text{the surface itself} \end{cases}$$

The sign convention can be flipped. Example — the **signed distance function** to a sphere of radius r : $f(p) = \|p\| - r$.



An implicit scalar field: green (positive) inside, red (negative) outside; the white boundary is the zero level-set, i.e. the surface.

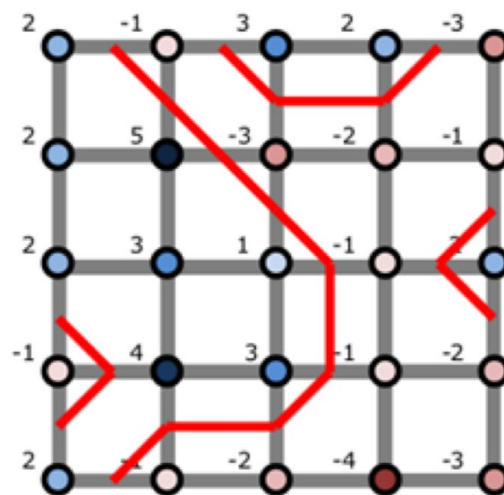
Marching Squares / Marching Cubes

Purpose: implicit $\rightarrow$ explicit — turn a scalar field f into an actual polygon mesh by finding where $f = 0$.

1. Sample f on a uniform grid.
2. Classify each grid corner by its sign (+/-). 4 corners $\rightarrow 2^4 = 16$ cases (2D); 8 corners $\rightarrow 2^8 = 256$ cases (3D), reducible to 15 by symmetry.
3. For each cell, find edge intersections — a sign change along an edge means the surface crosses it. Linearly interpolate the exact crossing point:

$$t = \frac{f_0}{f_0 - f_1}$$

4. For each cell, connect the intersection points using a precomputed lookup table (topology: which edges connect into which line segment or triangle).
5. Repeat for every cell.



Reconstructed contour lines from per-cell edge intersections.

Ambiguous cases

A diagonal sign change inside one cell can create 4 intersection points, connectable in two different topologically valid ways — the wrong choice creates a hole in the surface. Fix: evaluate f additionally at the cell center to disambiguate.

Hoppe's method — point cloud to implicit function

Pipeline: ICP $\rightarrow$ Hoppe (build implicit function) $\rightarrow$ Marching Squares/Cubes $\rightarrow$ triangle mesh.

Given a point cloud with normal vectors n_i :

1. Find the closest scan point q (KD-tree).
2. Compute the signed distance to the **tangential plane** of q : $f(p) = (p - q) \cdot n$.

Evaluate this on a uniformly sampled grid, then feed it into Marching Cubes.

- + Easy to implement; works well on uniformly sampled, low-noise data.
- – $f(p)$ is piecewise linear, which creates artifacts in undersampled regions.

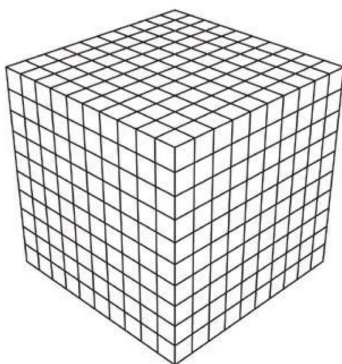
Improvement: compute the distance as a k -NN weighted blend of several nearby tangent-plane fits, instead of using only the single closest point:

$$f(p) = \frac{\sum_k w(\|q_k - p\|) d_k}{\sum_k w(\|q_k - p\|)} \quad (\text{smoother result})$$

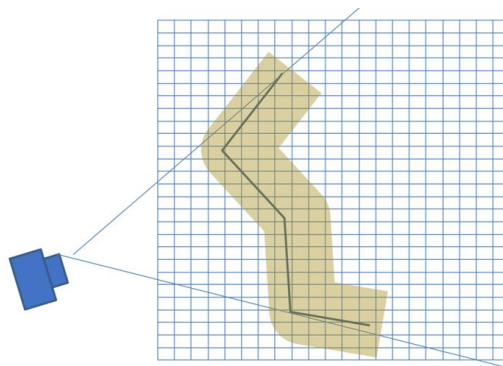
Truncated Signed Distance Fields (TSDF)

- Store the SDF as one number per voxel of a 3D voxel grid — *truncated*: only store values near the actual surface.
- Each depth image defines and updates a part of the surface.
- In the neighbourhood of the surface, set signed-distance values into the grid. If a voxel's value is already set from an earlier scan, merge the values — this smooths out noise.
- “Erase” the region between the camera and the surface (empty space, definitely not occupied).

Reconstruct the final mesh with Marching Cubes on the resulting TSDF grid.



3D voxel grid



Top-down view: measured surface vs. free space cleared by the camera ray